Ders 6 Genişletilmiş Notu: Uygulama-2 (Zar Olasılık Matrisi, Collatz, Fatura, Lab 1 Çözümü), realloc, Düzensiz Matris, Fonksiyondan Dizi Döndürme ve main Argümanları

Ders 6 Genişletilmiş Notu: Uygulama-2 (Zar Olasılık Matrisi, Collatz, Fatura, Lab 1 Çözümü), realloc, Düzensiz Matris, Fonksiyondan Dizi Döndürme ve main Argümanları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Birinci Soru: Üç Zarın Toplam Olasılık Matrisi
- 2. İkinci Soru: Collatz Dizisi
- 3. Üçüncü Soru: Toptancı Faturası
- 4. Lab 1 Çözümü: İki Matriste Ortak Sayılar
- 5. realloc: Ayrılmış Belleğin Boyutunu Değiştirmek
- 6. Düzensiz (Jagged) Matris
- 7. Fonksiyonlar: Sıra, Prototip ve Dizi Parametreleri
- 8. Fonksiyondan Dizi Döndürme
- 9. main Fonksiyonunun Argümanları

Dersin ilk yarısı uygulamadır: Arş. Grv. Begüm Özbay üç soru çözdü ve Lab 1'in çözümünü gösterdi. İkinci yarıda Dr. Ahmet Elbir ek ders olarak dinamik bellek konusunun kalan fonksiyonu `realloc`'u, düzensiz matrisi, fonksiyondan dizi döndürmeyi ve `main` argümanlarını anlattı.

> **Ders sonrası not:** Altyazı, kod yazılan bölümlerde çoğu zaman ekrandaki satırları vermiyor. 1-4. bölümlerdeki tam kodlar, anlatımdan ve kısa nottaki fonksiyon adlarından kuruldu. Bu nottaki bütün tam kodlar `gcc -Wall` ile uyarısız derlenip çalıştırıldı. Değişken adları kısa nottaki gibidir; ekrana yazılan mesajlar temsilîdir.

1. Birinci Soru: Üç Zarın Toplam Olasılık Matrisi

1.1 Problem

Üç zar aynı anda atılıyor ve üst yüzlere gelen sayılar toplanıyor. İki sütunlu bir matris tanımlanacak: birinci sütunda her olası toplam, ikinci sütunda o toplamın gerçekleşme olasılığı yer alacak. Matris dinamik bellekle oluşturulacak.

Temelde iki iş vardır: matrisi tanımlamak ve olasılığı hesaplamak. Asistan iki şart da koydu:

- **Birinci sütun benzersiz olmalı:** Aynı toplam değeri matriste tekrar etmemeli.
- **Olasılıklar elle yazılmamalı:** Değerler koda sabit (hard-coded) girilmeyecek. Döngü, kontrol ve aritmetik işlemlerle hesaplanacak.

1.2 Matrisi Oluşturan Fonksiyon

Dinamik bellek için `stdio.h`'nin yanında `stdlib.h` de eklenir, çünkü `malloc` ve `calloc` bu kütüphanededir. `initialize_mat` fonksiyonu satır sayısını (`nrow`) ve sütun sayısını (`ncolumn`) tam sayı olarak alır, belleği ayırır ve adresi döndürür.

İşlem iki adımdır. Önce satırları gösterecek `float` işaretçilerinden oluşan dizi için `malloc` ile yer ayrılır. Sonuç `NULL` ise ekrana "bellek ayrılmadı" yazılır ve `exit(1)` ile programdan çıkılır. Sonra `for` döngüsüyle her satır için `calloc` ile `ncolumn` kadar `float` ayrılır. Böylece her satır kendi `float` dizisine sahip olur.

Aynı döngüde birinci sütun da doldurulur. Üç zarın en küçük toplamı 1 + 1 + 1 = 3, en büyük toplamı 6 + 6 + 6 = 18'dir. Yani toplam 3 ile 18 arasında değişir. Döngü değişkeni `i` ise 0'dan başlar. Toplamı doğru satıra yerleştirmek için `mat[i][0] = i + 3` yazılır:

1. İlk turda `i = 0`, dolayısıyla `mat[0][0] = 0 + 3 = 3` olur.
2. İkinci turda `i = 1`, dolayısıyla `mat[1][0] = 1 + 3 = 4` olur.
3. Son turda `i = 15`, dolayısıyla `mat[15][0] = 15 + 3 = 18` olur (ara adım eklendi).

Fonksiyon `float **` döndürür, yani işaretçiyi gösteren bir işaretçi döndürür. Neden iki yıldız gerektiği önemlidir: İki boyutlu bir diziyi dinamik olarak yönetmek için, satırları gösteren bir işaretçi dizisine ihtiyaç vardır. O dizinin her elemanı da kendi satırının `float` dizisini gösterir.

> **Ders sonrası not:** Asistan bir yerde "pointer to pointer to pointer" dedi. `float **` için doğrusu "pointer to pointer", yani çift işaretçidir.

1.3 Olasılık Hesabı

`calc_prob` fonksiyonu matrisi parametre olarak alır. Her zar için bir döngü olacak şekilde iç içe üç `for` kurulur. `i`, `j` ve `k` zarların gelen değerlerini temsil eder.

Yazım sırasında asistan bir eksiği düzeltti: `i`, `j`, `k` zar değeri olduğu için en küçük 1, en büyük 6 olabilir. Bu yüzden üç döngüde de koşul `< 6` değil `<= 6` olmalıdır.

Asıl hesap en içteki döngüdedir. Önce `toplam = i + j + k` bulunur. Sonra bu toplamın satırındaki olasılığa bir kombinasyonun payı eklenir:

```
toplam = i + j + k;
mat[toplam - 3][1] += 1.0 / 216;
```

Her zar 6 değer alabildiği için toplam $6 \times 6 \times 6 = 216$ kombinasyon vardır. Bir kombinasyonun olasılığı $\frac{1}{6} \cdot \frac{1}{6} \cdot \frac{1}{6} = \frac{1}{216}$'dır. Döngü bütün kombinasyonları gezer ve her birinde ilgili toplamın olasılığını $\frac{1}{216}$ artırır. Asistan mantığı şu örneklerle gösterdi:

- **Toplam 3:** Zarlar 1, 1, 1 gelince `toplam - 3 = 0` olur, yani `mat[0][1]`'e ekleme yapılır. 3 toplamı başka türlü elde edilemez, bu yüzden olasılık $\frac{1}{216}$'da kalır.
- **Toplam 4:** 1-1-2 ve 1-2-1 gibi birden çok kombinasyonla gelebilir. Her biri aynı satıra bir pay daha ekler.
- **Toplam 5:** 1-2-2 ve 3-1-1 gibi kombinasyonlarla gelebilir.

> **Ders sonrası not:** Asistan, toplam 5 için "atıyorum 5 farklı şekilde" diyerek varsayımsal bir sayı söyledi. Python ile hesaplandı: toplam 5'i veren 6 kombinasyon vardır (1-1-3, 1-3-1, 3-1-1, 1-2-2, 2-1-2, 2-2-1). Toplam 4'ü veren 3 kombinasyon vardır (1-1-2, 1-2-1, 2-1-1).

> **Ek bilgi (derste anlatılmadı):** Koddaki `1.0 / 216` ifadesinde `1.0` yazılması gerekir. `1 / 216` yazılırsa iki tam sayı bölünür ve sonuç 0 olur, olasılıklar hep 0 kalır. gcc ile denendi: `1/216` 0, `1.0/216` 0.004630 verdi.

1.4 Belleği Temizleme, Yazdırma ve main

`free_mat` önce döngüyle her satırı, sonra satır dizisinin kendisini serbest bırakır. `print_probs` ise her satırı gezerek toplamı ve olasılığı yan yana yazar. Yan sütuna geçmek için `\t`, alt satıra geçmek için `\n` kullanılır.

`main` içinde satır sayısı 16, sütun sayısı 2 verilir. 3 ile 18 arasında $18-3+1=16$ farklı toplam olduğu için 16 satır gerekir (ara adım eklendi). Sütunlardan biri toplamı, diğeri olasılığı tutar. Çağrı sırası şöyledir:

1. `initialize_mat` ile matris oluşturulur.
2. `print_probs` ile yazdırılır. Hesap henüz yapılmadığı için olasılıklar 0 görünür, çünkü `calloc` belleği sıfırlar. Asistan bunu özellikle göstermek istedi.
3. `calc_prob` ile olasılıklar hesaplanır.
4. `print_probs` ile tekrar yazdırılır.
5. `free_mat` ile bellek boşaltılır ve `return 0` ile bitirilir.

   Ek bilgi (derste anlatılmadı): Birinci sütun döngüde `i + 3` ile yazıldığı için her toplam tam bir kez bulunur, benzersizlik şartı böyle sağlanır. `free_mat`'ta satır dizisi önce serbest bırakılsaydı, satırların adresleri kaybolurdu; bu yüzden önce satırlar serbest bırakılır.

Derleme sırasında sınıfla birlikte birkaç yazım hatası bulundu: bir yerde `float` işaretçi tür dönüşümünün parantezi eksikti, `print` fonksiyonunda `for` koşulu `i < nrow` yerine eksik yazılmıştı, parametre adı `nrow` iken döngüde başka ad kullanılmıştı, `ncolumn` da yanlış yazılmıştı.

Tür dönüşümündeki hatayı bir öğrenci fark etti: `float` tür adının parantez içine alınması gerekiyordu.

1.5 Tam Kod ve Çıktı

```c
#include <stdio.h>
#include <stdlib.h>

float **initialize_mat(int nrow, int ncolumn) {
    int i;
    float **mat = (float **) malloc(nrow * sizeof(float *));
    if (mat == NULL) {
        printf("Bellek ayrilamadi\n");
        exit(1);
    }
    for (i = 0; i < nrow; i++) {
        mat[i] = (float *) calloc(ncolumn, sizeof(float));
        if (mat[i] == NULL) {
            printf("Bellek ayrilamadi\n");
            exit(1);
        }
        mat[i][0] = i + 3;
    }
    return mat;
}

void calc_prob(float **mat) {
    int i, j, k, toplam;
    for (i = 1; i <= 6; i++) {
        for (j = 1; j <= 6; j++) {
            for (k = 1; k <= 6; k++) {
                toplam = i + j + k;
                mat[toplam - 3][1] += 1.0 / 216;
            }
        }
    }
}

void print_probs(float **mat, int nrow) {
    int i;
    printf("Toplam\tOlasilik\n");
    for (i = 0; i < nrow; i++) {
```

```
        printf("%.0f\t%f\n", mat[i][0], mat[i][1]);
    }
}

void free_mat(float **mat, int nrow) {
    int i;
    for (i = 0; i < nrow; i++) {
        free(mat[i]);
    }
    free(mat);
}

int main(void) {
    int nrow = 16, ncolumn = 2;
    float **mat = initialize_mat(nrow, ncolumn);

    print_probs(mat, nrow);
    calc_prob(mat);
    print_probs(mat, nrow);

    free_mat(mat, nrow);
    return 0;
}
```

Birinci yazdırmada bütün olasılıklar 0.000000'dır. İkinci yazdırmada çıktı şöyledir (gcc ile çalıştırıldı). Kombinasyon sayısı ve kesir sütunları python ile hesaplandı:

Toplam	Kombinasyon sayısı	Olasılık (kesir)	Çıktı
3	1	1/216	0.004630
4	3	3/216	0.013889
5	6	6/216	0.027778
6	10	10/216	0.046296
7	15	15/216	0.069444
8	21	21/216	0.097222
9	25	25/216	0.115741
10	27	27/216	0.125000
11	27	27/216	0.125000
12	25	25/216	0.115741
13	21	21/216	0.097222
14	15	15/216	0.069444
15	10	10/216	0.046296
16	6	6/216	0.027778
17	3	3/216	0.013889
18	1	1/216	0.004630

Kombinasyon sayılarının toplamı 216'dır, yani olasılıkların toplamı 1'dir (python ile hesaplandı).

2. İkinci Soru: Collatz Dizisi

2.1 Problem

Şu fonksiyon veriliyor:

$$f(n) = \begin{cases} 3n+1 & n \bmod 2 = 1 \\ n/2 & n \bmod 2 = 0 \end{cases}$$

$n \bmod 2 = 1$ sayının tek, $n \bmod 2 = 0$ sayının çift olduğunu gösterir. Bu fonksiyon matematikteki Collatz problemine aittir. Problem, hangi pozitif tam sayıdan başlanırsa başlansın,

fonksiyon tekrar tekrar uygulandığında dizinin 4, 2, 1 döngüsüne gireceğini ve sonunda 1'e ulaşacağını iddia eder. Asistan bu iddianın günümüzde hâlâ tartışıldığını söyledi.

> **Ders sonrası not:** Bu iddia büyük olasılıkla hâlâ kanıtlanmamıştır, yani açık bir problemdir.

Sorudaki örnek 13'tür. 13 tek olduğu için $3 \cdot 13 + 1 = 40$ bulunur. 40 çift olduğu için $40/2 = 20$ bulunur. Devamı aynı kuralla ilerler (ara adımlar eklendi, python ile hesaplandı):

$13 \to 40 \to 20 \to 10 \to 5 \to 16 \to 8 \to 4 \to 2 \to 1$

Programdan istenenler şunlardır:

1. Kullanıcıdan pozitif bir `n` tam sayısı okunur.
2. `calloc` ile 2 elemanlık bir dizi oluşturulur.
3. `f`, 1 elde edilene kadar tekrar tekrar uygulanır (`while` ile) ve her değer diziye eklenir.
4. Dizi dolunca `realloc` ile boyutu iki katına çıkarılır.
5. Dizinin elemanları ve kapladığı alan ekrana yazılır.
6. Ayrılan alan `free` ile boşaltılır.

2.2 Kodun Kurulması

Fonksiyon, `n % 2 == 0` kontrolüyle yazılır: çiftse `n / 2`, değilse `3 * n + 1` döner. `main`'de önce kullanıcıdan sayı alınır. Sıfır ya da negatif değer girilirse kullanıcıya yeni bir değer girme hakkı verilir.

Sonra `calloc` ile `A` dizisi açılır. Başlangıç boyutu `current_length` değişkeninde (2) tutulur. Girilen sayı `A[0]`'a yazılır ve `i` değişkeni 1 yapılır. `while (n != 1)` döngüsünde `n`'in yeni değeri hesaplanır, dizinin `i`. elemanına yazılır, `i` bir artırılır. `i`, `current_length`'e eşit olduğunda dizi dolmuştur: `current_length` iki katına çıkarılır ve `realloc` ile bellek bu yeni boyuta genişletilir.

Yazdırma aşamasında `i`'nin son değeri eleman sayısıdır, çünkü `i` her eklemede bir artmıştır. Bu yüzden `j` değişkeni 0'dan `i`'ye kadar gider.

Derlemede şu hatalar bulundu: `n` tanımlanmamıştı, kontrolde atama (=) yerine eşitlik (==) gerekiyordu, `main`'in sonunda `return 0` yoktu, bir yerde `printf` yerine `print` yazılmıştı ve `free` eksikti.

Soru: Fonksiyonu `main`'in altına yazdığımızda imzasını `main`'in üstünde tanımlamamız gerekmiyor mu? **Cevap:** Gerekiyor. Üste prototip eklenerek düzeltildi.

2.3 Tam Kod

```c
#include <stdio.h>
#include <stdlib.h>

int f(int n);

int main(void) {
    int n, i, j;
    int current_length = 2;
    int *A;

    do {
        printf("Pozitif bir tam sayi giriniz: ");
        scanf("%d", &n);
    } while (n <= 0);

    A = (int *) calloc(current_length, sizeof(int));
    A[0] = n;
    i = 1;

    while (n != 1) {
        n = f(n);
```

```c
        A[i] = n;
        i++;
        if (i == current_length) {
            current_length = current_length * 2;
            A = (int *) realloc(A, current_length * sizeof(int));
        }
    }

    for (j = 0; j < i; j++) {
        printf("%d ", A[j]);
    }
    printf("\nEleman sayisi: %d, ayrilan alan: %d byte\n",
           i, (int) (current_length * sizeof(int)));

    free(A);
    return 0;
}

int f(int n) {
    if (n % 2 == 0) {
        return n / 2;
    } else {
        return 3 * n + 1;
    }
}
```

2.4 Deneme: 8

Asistan önce basit olduğu için 8'i girdi (eleman sayısı ve kapasite adımları gcc ile çalıştırılıp doğrulandı):

Adım	n	Yapılan	i	current_length
Başlangıç	8	`A[0] = 8`	1	2
1	4	$8/2$, `A[1] = 4`	2	$2 \to 4$ (dizi doldu)
2	2	$4/2$, `A[2] = 2`	3	4
3	1	$2/2$, `A[3] = 1`	4	$4 \to 8$ (dizi doldu)

`n` 1 olunca döngü durur. Çıktı 8, 4, 2, 1 dizisidir. Eleman sayısı 4'tür. Bu kodda ayrılan alan $8 \times 4 = 32$ byte olur, çünkü `int` 4 byte'tır (gcc ile denendi).

Asistan ardından daha büyük bir sayı denedi. 7 için adımları anlattı: 7 tek olduğu için $3 \cdot 7 + 1 = 22$, 22 çift olduğu için $22/2 = 11$ bulunur ve bu şekilde devam edilir.

> **Ders sonrası not:** Python ile hesaplandı: 7 için dizi 17 elemandır (7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1). 27 için dizi 112 elemandır ve en büyük değer 9232'dir. 13 için dizi 10 elemandır.
>
> **Ders sonrası not:** Uygulamada `A = (int *) realloc(A, ...)` yazıldı ve `NULL` kontrolü yapılmadı. Hocanın 5. bölümde anlattığı geçici işaretçi yöntemi daha güvenlidir. Kısa nota göre elemanlar ekrana aralarında boşluk olmadan yazdırılıyordu; yukarıdaki kodda `"%d "` kullanıldı.

3. Üçüncü Soru: Toptancı Faturası

3.1 Problem

Bir toptancı soda, meyve suyu, kola ve su olmak üzere dört çeşit meşrubat satıyor. Sipariş edilen ürünler için fatura oluşturulacak. Faturada her ürün için ürün sayısı, ürün adı, birim fiyat ve tutar bulunacak. Tutar, birim fiyat ile adedin çarpımıdır. Faturanın en altında da genel toplam yer alacak.

Kullanıcı her üründen kaç koli aldığını girer. Birim fiyatlar kodda tanımlıdır: 10, 15, 20 ve 5 TL.

> **Ders sonrası not:** Altyazıda bir yerde fiyatlar "10, 15, 25" diye geçiyor. Soru metni ve çalıştırma anlatımı 10, 15, 20, 5 olduğu için doğrusu budur.

3.2 Fonksiyonlar

Üç fonksiyon yazılır:

- **`generate_invoice`:** Miktarları, ürün adlarını ve birim fiyatları parametre alır, fatura satırlarını oluşturup döndürür.
- **`print_invoice`:** Fatura satırlarını döngüyle tek tek ekrana yazar.
- **`free_invoice`:** Aynı döngü mantığıyla her satırı serbest bırakır, sonra satır dizisini serbest bırakır.

`generate_invoice` içinde `total_amount` 0 ile başlatılır. Fatura satırlarını tutacak `invoice_lines` için `malloc` ile yer ayrılır. Ürün sayısı `#define MAX_PRODUCTS 4` ile, bir satırın en büyük uzunluğu `#define MAX_STRING_LENGTH 100` ile tanımlanır.

Döngü her ürün için (`i` = 0, 1, 2, 3) şunları yapar:

1. Ürünün tutarı hesaplanır: `amount = quantities[i] * unit_prices[i]`.
2. Tutar genel toplama eklenir: `total_amount += amount`.
3. Bu satır için `malloc(MAX_STRING_LENGTH)` ile yer ayrılır.
4. Miktar, ad, birim fiyat ve tutar `sprintf` ile satıra yazılır.

Döngüden sonra son satır olarak genel toplam da aynı şekilde yazılır ve `invoice_lines` döndürülür.

`main` içinde ürün adları ve birim fiyatlar dizi olarak tanımlanır. Miktarlar kullanıcıdan alınacağı için `quantities` dizisi `MAX_PRODUCTS` boyutunda açılır. Kullanıcıya `%s` ile ürünün adı gösterilerek ("kola için giriniz" gibi) değer istenir. Sonra sırasıyla `generate_invoice`, `print_invoice` ve `free_invoice` çağrılır.

3.3 Döngü İçinde Değişken Tanımı

Canlı derlemede hata alındı ve kod sınıfta tamamlanamadı. Asistan hazır programı çalıştırarak çıktıyı gösterdi. Sorunu hoca buldu: `int amount` döngünün içinde tanımlanmıştı.

> **Hocanın vurgusu:** `amount` değişkeni döngü içinde tanımlanmaz, yukarıda tanımlanır ve aşağıda kullanılır. Bu C++'tan gelen bir alışkanlıktır. Başka bir zaman çalışmayabilir, buna dikkat edin.
>
> **Ders sonrası not:** Bu uyarı eski C standardı için geçerlidir. gcc ile denendi: `-std=c90 -pedantic-errors` ile `for (int j...)` derleme hatası verir, `-std=c99` ile geçerlidir. C99'dan beri döngü içinde tanım yapılabilir. Yine de bu derste kodlar hocanın tercihine uygun yazıldı.

3.4 Tam Kod ve Örnek

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX_PRODUCTS 4
#define MAX_STRING_LENGTH 100

char **generate_invoice(int quantities[], char *product_names[], int unit_prices[]) {
    int total_amount = 0;
    int i, amount;
    char **invoice_lines = (char **) malloc((MAX_PRODUCTS + 1) * sizeof(char *));

    for (i = 0; i < MAX_PRODUCTS; i++) {
        amount = quantities[i] * unit_prices[i];
```

```
        total_amount += amount;
        invoice_lines[i] = (char *) malloc(MAX_STRING_LENGTH);
        sprintf(invoice_lines[i], "%d koli %s, birim fiyat: %d TL, tutar: %d TL",
                quantities[i], product_names[i], unit_prices[i], amount);
    }

    invoice_lines[MAX_PRODUCTS] = (char *) malloc(MAX_STRING_LENGTH);
    sprintf(invoice_lines[MAX_PRODUCTS], "Toplam tutar: %d TL", total_amount);
    return invoice_lines;
}

void print_invoice(char **invoice_lines) {
    int i;
    for (i = 0; i <= MAX_PRODUCTS; i++) {
        printf("%s\n", invoice_lines[i]);
    }
}

void free_invoice(char **invoice_lines) {
    int i;
    for (i = 0; i <= MAX_PRODUCTS; i++) {
        free(invoice_lines[i]);
    }
    free(invoice_lines);
}

int main(void) {
    char *product_names[MAX_PRODUCTS] = {"soda", "meyve suyu", "kola", "su"};
    int unit_prices[MAX_PRODUCTS] = {10, 15, 20, 5};
    int quantities[MAX_PRODUCTS];
    char **invoice_lines;
    int i;

    for (i = 0; i < MAX_PRODUCTS; i++) {
        printf("%s icin koli sayisini giriniz: ", product_names[i]);
        scanf("%d", &quantities[i]);
    }

    invoice_lines = generate_invoice(quantities, product_names, unit_prices);
    print_invoice(invoice_lines);
    free_invoice(invoice_lines);
    return 0;
}
```

Örnekteki girdi 3 soda, 2 meyve suyu, 5 kola, 10 sudur. Girdi kısa nottan alındı, sonuç python ve gcc ile doğrulandı:

- Soda: $3 \times 10 = 30$ TL
- Meyve suyu: $2 \times 15 = 30$ TL
- Kola: $5 \times 20 = 100$ TL
- Su: $10 \times 5 = 50$ TL
- Toplam: $30 + 30 + 100 + 50 = 210$ TL

4. Lab 1 Çözümü: İki Matriste Ortak Sayılar

4.1 Problem

$2 \times n$ ve $2 \times m$ boyutunda iki matris vardır. İlk satırda benzersiz tam sayılar, ikinci satırda bu sayıların karşılık değerleri bulunur. Program, iki matrisin ilk satırlarındaki ortak sayıları bulacak ve karşılık değerlerini toplayacak.

Örnek şudur:

	1. matris	2. matris
İlk satır	1, 5	2, 3, 5
İkinci satır	7, 2	4, 8, 1

İlk satırlardaki ortak sayı 5'tir. 1. matriste 5'in karşılık değeri 2, 2. matriste 1'dir. Sonuç $2+1=3$ olur.

4.2 Çözümün Adımları

Programın başında `stdio.h` ve Türkçe karakter desteği için `locale.h` eklenir, en büyük matris boyutu `#define` ile belirlenir. `main`'de `setlocale` ile Türkçe karakter desteği açılır. `matris1` ve `matris2` tanımlanır. `n` ve `m` boyutları, `i` ve `j` döngü değişkenleridir.

1. Kullanıcı `n`'i girer, sonra 1. matrisin ilk satırını girer.
2. Her sayıda benzersizlik kontrolü yapılır. Bayrak (flag) değişkeni 0 ile başlatılır, sayı `scanf` ile alınır, `for` döngüsüyle daha önce girilen sayılara bakılır. Aynı sayı varsa bayrak 1 olur. `if` ile bayrağa bakılır ve 1 ise "matrisin ilk satırı benzersiz olmalıdır" uyarısı verilir.
3. Kullanıcı ikinci satırı, yani karşılık değerlerini girer.
4. 2. matris için `m` ile aynı adımlar tekrarlanır.
5. Ortak sayılar iç içe iki döngüyle aranır: dış döngü 1. matrisin ilk satırını (`n`'e kadar), iç döngü 2. matrisin ilk satırını (`m`'e kadar) gezer. İki değer eşitse sayı `ortak_sayi`'ya, karşılık değerleri `deger1` ve `deger2`'ye alınır, `toplam = deger1 + deger2` hesaplanır ve `printf` ile yazılır.

```c
#include <stdio.h>
#include <locale.h>

#define MAX 100

int main(void) {
    int matris1[2][MAX], matris2[2][MAX];
    int n, m, i, j, bayrak;
    int ortak_sayi, deger1, deger2, toplam;

    setlocale(LC_ALL, "Turkish");

    printf("1. matrisin n degerini giriniz: ");
    scanf("%d", &n);
    printf("1. matrisin ilk satirini giriniz:\n");
    for (i = 0; i < n; i++) {
        do {
            bayrak = 0;
            scanf("%d", &matris1[0][i]);
            for (j = 0; j < i; j++) {
                if (matris1[0][j] == matris1[0][i]) {
                    bayrak = 1;
                }
            }
            if (bayrak == 1) {
                printf("Matrisin ilk satiri benzersiz olmalidir.\n");
            }
        } while (bayrak == 1);
    }
    printf("1. matrisin ikinci satirini giriniz:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &matris1[1][i]);
    }
```

```
    printf("2. matrisin m degerini giriniz: ");
    scanf("%d", &m);
    printf("2. matrisin ilk satirini giriniz:\n");
    for (i = 0; i < m; i++) {
        do {
            bayrak = 0;
            scanf("%d", &matris2[0][i]);
            for (j = 0; j < i; j++) {
                if (matris2[0][j] == matris2[0][i]) {
                    bayrak = 1;
                }
            }
            if (bayrak == 1) {
                printf("Matrisin ilk satiri benzersiz olmalidir.\n");
            }
        } while (bayrak == 1);
    }
    printf("2. matrisin ikinci satirini giriniz:\n");
    for (i = 0; i < m; i++) {
        scanf("%d", &matris2[1][i]);
    }

    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            if (matris1[0][i] == matris2[0][j]) {
                ortak_sayi = matris1[0][i];
                deger1 = matris1[1][i];
                deger2 = matris2[1][j];
                toplam = deger1 + deger2;
                printf("Ortak sayi: %d, deger1: %d, deger2: %d, toplam: %d\n",
                       ortak_sayi, deger1, deger2, toplam);
            }
        }
    }
    return 0;
}
```

Derste girilen değerlerle ($n = 2$: 1, 5 ve 7, 2; $m = 3$: 2, 3, 5 ve 4, 8, 1) çıktıda ortak sayı 5, `deger1` 2, `deger2` 1, `toplam` 3 olur (gcc ile denendi).

5. realloc: Ayrılmış Belleğin Boyutunu Değiştirmek

Hoca dinamik bellek fonksiyonlarının kalanıyla devam etti. `malloc` ve `calloc` konuşulmuştu, `free` zaten kullanılıyordu. Geriye `realloc` kaldı: Program çalışırken bellek ihtiyacı değişirse belleği artırıp azaltmayı sağlayan fonksiyon budur.

5.1 Çalışma Mantığı

`malloc` ve `calloc` bir yer ayırıp işaretçi döndürüyordu, bu işaretçi tür dönüşümüyle istenildiği gibi kullanılıyordu. `realloc`'ta durum biraz farklıdır: Elde daha önce `malloc` veya `calloc` ile oluşturulmuş bir işaretçi olmalıdır. Fonksiyona bu eski bilgi ve yeni boyut verilir, işletim sisteminden yeniden yer istenir:

```
realloc(eski_isaretci, yeni_boyut);
```

Hoca bunu belleği çizerek anlattı. Çizilen şey bütün RAM değil, prosese atanmış bellek bölgesidir. `malloc` ile oluşturulan yer mutlaka heap'tedir. Hoca heap'in aşağıdan yukarıya, stack'in yukarıdan aşağıya doğru dolduğunu hatırlattı. Elimizde n uzunluğunda bir bölge olsun ve içinde 1, 2, 3, 4 verileri bulunsun. Bundan $2n$ uzunluğunda bir yer isteyelim. Üç ihtimal vardır:

1. **Bulunduğu yerden itibaren yer varsa:** Bölgenin devamında yeterli boşluk varsa uğraşmaya gerek yoktur. Alan olduğu yerde genişletilir.
2. **Başka yerde yer varsa:** Yine heap içinde $2n$ uzunluğunda başka bir bölge aranır. Bulunursa eski veriler oraya birebir kopyalanır. Eski bölge otomatik olarak serbest bırakılır, orada yalnızca çöp değerler kalır. Bu sırada heap'i genişletme gibi ek işletim sistemi optimizasyonları da yapılır.
3. **Hiç yer yoksa:** İstek çok büyükse fonksiyon "istediğin kadar yer veremiyorum" der ve hata döndürür. Daha önce ayrılmış bölge ise dokunulmadan (intact) olduğu gibi kalır.

Hoca, arka planda aslında gizli bir `malloc` çalıştığını, `realloc`'un `malloc` üzerine kurulmuş üst seviye bir fonksiyon gibi düşünülebileceğini söyledi.

> **Ders sonrası not:** Hoca, yer bulunamayınca `realloc`'un "negatif bir değer" döndüreceğini söyledi; doğrusu `NULL` döndürmesidir. Hocanın kodu da `NULL` kontrolü yapar. gcc ile denendi: çok büyük bir istekte `realloc` `NULL` döndürdü ve eski işaretçi kullanılabilir kaldı.

5.2 Örnek: Sürekli Büyüyen Dizi

Hoca n elemanlı bir dizi kurdu ve kullanıcı istedikçe büyüten bir program yazdı. En az bir kez büyütme denensin diye `do-while` kullandı. Her turda yeni bir `m` değeri alınır, yeni boyut eski boyutun üzerine eklenir. Örneğin önce `n`'e 100, sonra `m`'ye 50 verilirse yeni boyut 150 olur. Toplam kümülatif olarak büyür: Yeni `n`, `n + m` olur, bir sonraki turda üstüne yine eklenir.

`realloc` çağrısının ikinci parametresi byte cinsinden boyuttur, bu yüzden `sizeof(int) * (n + m)` yazılır. Kullanıcıdan karakter okurken, önceki Enter tuşu okunmasın diye `%c`'nin önüne bir boşluk konabileceğini ya da `fflush(stdin)` yazılabileceğini de hatırlattı.

> **Ders sonrası not:** Hoca `fflush(stdin)` seçeneğini de andı. Standart C'de `fflush` büyük olasılıkla yalnızca çıkış akışları için tanımlıdır, `stdin` için davranış Windows'a özgüdür. Taşınabilir yol, hocanın ilk söylediği `scanf(" %c", &c)` biçimidir.

Hoca değerleri giderek büyüttü: 500, 5000, 5000, 50000, 50000, 500000 ve daha büyükleri. Şöyle gözlemler yaptı:

- İlk büyütmede boyut 520 oldu.
- Büyük isteklerde `NULL` dönmüyordu, çünkü bilgisayarın RAM'i fazlaydı.
- Bazı çok büyük isteklerde program bir süre bekledi ve sonra yer buldu. Hocaya göre bu bekleme büyük olasılıkla eski belleğin başka bir yere birebir kopyalanmasından kaynaklanıyordu.
- İsteği 100 katına çıkarınca `Error` mesajı alındı ve program `-2` ile çıktı. Yeterince yer bulunamamıştı.

Hata mesajlarını ayırt edebilmek için ilk `malloc` hatasına `Error 1`, `realloc` hatasına `Error 2` adı verildi. Dönüş değerleri de `-1` ve `-2` yapıldı.

5.3 Mantıksal Hata: Sonucu Aynı İşaretçiye Atamak

Hoca bu kodu önce şöyle yazdı ve sınıfa bir soru sordu: Burada tekrar `array`'i kullanmak doğru mu?

```
array = (int *) realloc(array, sizeof(int) * (n + m));
if (array == NULL) {
    perror("Error 2");
    return -2;
}
```

Hata alındığında programın kapanmadığını, yalnızca daha fazla yer alınamadığı için devam etmek istediğimizi düşünelim. `array` önceki turlarda belli bir boyuta kadar gelmiş olsun. Son `realloc` `NULL` döndürünce `array`'in değeri de `NULL` olur. O ana kadar ayrılmış bölgenin adresi kaybolur. Hocanın ifadesiyle, en son elde edilen `NULL` yüzünden hepsi "uçtu gitti".

5.4 Güvenli Kullanım

Çözüm, sonucu önce geçici bir işaretçiye (`tmp`) almaktır:

```
tmp = (int *) realloc(array, sizeof(int) * (n + m));
if (tmp == NULL) {
    perror("Error 2");
    state = 0;
} else {
    array = tmp;
}
```

Adımlar şöyledir. `array` ve `tmp` stack'teki iki işaretçi değişkenidir.

1. Başlangıçta `array` heap'teki eski bölgeyi gösterir.
2. `realloc` sonucu `tmp`'ye alınır. `tmp` artık yeni bölgeyi gösterir, `array` hâlâ eski bölgeyi gösterir ve eski veriler yaşamaktadır.
3. `tmp` `NULL` ise yapılacak bir şey yoktur, hata verilir. `array` duruyor olduğu için hiçbir şey kaybolmaz.
4. `tmp` `NULL` değilse istenen yer alınmıştır, `array = tmp` yazılır. Artık ikisi de yeni bölgeyi gösterir ve `array`'in eski bölgeyle bağlantısı kalmaz. Eski bölge zaten otomatik serbest bırakılmıştır.

   Hocanın vurgusu: `realloc` işlemini güvenli şekilde yapmanın yolu budur: Sonucu önce geçici değişkende kontrol edin, başarılıysa asıl işaretçiye atayın.

Döngüden çıkışı yönetmek için `state` adında bir değişken eklendi. Başlangıçta 1'dir, yer alınamadığında 0 olur. Döngü `state == 1` olduğu ve kullanıcı devam etmek istediği sürece döner. Kullanıcı çıkmak istemese de yer verilmediği için çıkılmış olabilir. Her iki durumda da döngüden sonra `array` geçerlidir: Beşinci elemanına gidilebilir, `for` ile gezilebilir, elemanlar yazdırılabilir.

Tam kod aşağıdadır. Hocanın yazdırma fonksiyonu `print_list` 6. bölümde yazıldı, sonra buraya da eklendi. `malloc` ile açılan alan çöp değerler içerebileceği için dizi başta temiz bir içerikle doldurulur. Sonradan büyütülen kısımda çöp değer görülebilir.

```
#include <stdio.h>
#include <stdlib.h>

void print_list(int *list, int size) {
    int i;
    for (i = 0; i < size; i++) {
        printf("%d ", list[i]);
    }
    printf("\n");
}

int main(void) {
    int n, m, i;
    int state = 1;
    char c = 'Y';
    int *array, *tmp;

    printf("n degerini giriniz: ");
    scanf("%d", &n);

    array = (int *) malloc(n * sizeof(int));
    if (array == NULL) {
        perror("Error 1");
        return -1;
    }
    for (i = 0; i < n; i++) {
        array[i] = i;
    }
```

```
    print_list(array, n);

    do {
        printf("m degerini giriniz: ");
        scanf("%d", &m);

        if (n + m <= 0) {
            printf("Yeni boyut pozitif olmali.\n");
            state = 0;
        } else {
            tmp = (int *) realloc(array, sizeof(int) * (n + m));
            if (tmp == NULL) {
                perror("Error 2");
                state = 0;
            } else {
                array = tmp;
                n = n + m;
                print_list(array, n);
                printf("Devam etmek icin Y giriniz: ");
                scanf(" %c", &c);
            }
        }
    } while (state == 1 && (c == 'Y' || c == 'y'));

    free(array);
    return 0;
}
```

Ders sonrası not: Yukarıdaki kodda `n + m <= 0` denetimi ders sonrasında eklendi. Gerekçesi 5.5'tedir.

5.5 Dizi Küçültme ve Sıfır Boyut

Hoca bir ayrıntıyı sonradan ekledi: `realloc` küçültmek için de kullanılabilir. `m`'ye negatif değer girilirse yeni boyut azalır. Örnekleri şunlardı (python ile hesaplandı):

Başlangıç `n`	Girilen `m`	Yeni `n`	Sonuç
100	-100	0	Alan serbest kaldı, `NULL` döndü, `Error 2`
5	3	8	8 değer yazıldı
8	-2	6	6'ya inmesi beklendi; denemede `Y` girilmediği için görülemedi
7	-2	5	Debugger'da 5 görüldü
3	2	5	Büyüdü
5	-1	4	Dizi küçüldü
4	-4	0	`NULL` döndü, program kapandı

Hoca satır 15'ten sonra `n`'in 0 olduğunu gösterdi. Yeni boyut 0 olduğunda alan gerçek anlamda serbest bırakılmış olur, işaretçi `NULL` değerine düşer. Bu yüzden `Error 2`'ye gidilmişti. Bir denemede beklenmedik hata alınınca hoca kesme noktası (breakpoint) koydu ve adım adım ilerledi. Debugger'da `tmp` ile `array`'in aynı adresi gösterdiği, istenen yer verilebildiği için `NULL` olmadığı görüldü. Hatanın sebebi, devam etmek için `Y` girilmemesiydi.

Ders sonrası not: Derste `realloc(array, 0)` `NULL` döndürdü, ama bu davranış büyük olasılıkla kütüphaneye bağlıdır ve C23'te büyük olasılıkla tanımsız davranış sayılır. Burada önemli bir tuzak vardır: Alan serbest kalmış olsa da `array` eski adresi

tutmaya devam eder. Döngüden sonra `free(array)` çağrılırsa aynı alan iki kez serbest bırakılır. gcc ile AddressSanitizer açık denendi, `n + m == 0` durumunda "double-free" hatası alındı. Bu yüzden alanı serbest bırakmak için `realloc(p, 0)` yerine `free` kullanın.

6. Düzensiz (Jagged) Matris

6.1 Kavram

Normal bir matriste satır sayısı verilir ve bütün satırların sütun sayısı eşittir. Bazen her satırın sütun sayısının farklı olması gerekir: bir satır 3 sütunlu, diğeri 2 sütunlu, bir sonraki daha uzun olabilir. Buna düzensiz matris denir, İngilizcesi jagged ya da ragged'dır. Bazı dillerde bu yapı hazır gelir, ama C'de doğrudan böyle bir veri yapısı yoktur.

6.2 Kurulum

Satır sayısı `n` kullanıcıdan alınır. Normal bir matriste olduğu gibi `n` tane işaretçiyi tutacak ikinci dereceden bir işaretçi gerekir. Fazladan bir şey daha gerekir: Her satırın kaç sütunlu olduğunu tutan yardımcı bir dizi (`array`).

Adımlar şöyledir:

1. `array = malloc(n * sizeof(int))` ile yardımcı dizi açılır.
2. Dizi `array[i] = rand() % n + 1` ile doldurulur. `+ 1` sütun sayısının hiçbir zaman 0 gelmemesi içindir. Başta `srand(time(NULL))` çağrılır. Sütun sayıları kullanıcıdan da alınabilir.
3. `matrix = malloc(n * sizeof(int *))` ile satır işaretçileri dizisi açılır. `sizeof` içine `int *` yazılır, çünkü burada adreslerin adresi tutulur. Bir adres 8 byte yer kaplar.
4. Her satır için `matrix[i] = malloc(array[i] * sizeof(int))` ile o satırın gerçek belleği ayrılır. Normal matristen farkı buradadır: Sütun sayısı sabit değil, `array[i]`'dir.
5. Matris doldurulur.
6. Bellek serbest bırakılır.

Hoca bu kodda `NULL` kontrollerini yapmadığını, normalde `matrix` ve `matrix[i]` için az önceki kontrollerin yapılması gerektiğini, ama konu o olmadığı için geçtiğini belirtti.

> **Ders sonrası not:** Hoca adresin 8 byte olmasını açıklarken "8 byte 32 demek, 2^{32} de 4 giga adresler" dedi. Doğrusu şöyledir (python ile hesaplandı): 8 byte = $8 \times 8 = 64$ bittir. $2^{32} = 2^2 \cdot 2^{30}$, yani 4 GB, 4 byte'lık (32 bit) işaretçilerin sınırıdır. 8 byte'lık işaretçi teorik olarak 2^{64} adres gösterebilir. gcc ile `sizeof(int *)` 8 çıktı.

6.3 Doldurma: İç Döngünün Sınırı

Dış döngü `i = 0`'dan `n`'e kadar gider. Satır sayısı sabit olduğu için sınır `n`'dir.

Soru: İç döngüde `j` kaça kadar gidecek? **Cevap:** Normal matristeki gibi sabit bir `m` yazılamaz, çünkü sütun sayısı satıra göre değişir. Sınır, o satırın sütun sayısını tutan `array[i]`'dir. `array[j]` değil, çünkü bilgi satıra aittir ve satırı gösteren `i`'dir.

Hoca önce `array[j]` diyerek sınıfı denedi, bir öğrenci `array[i]` gerektiğini söyledi.

> **Hocanın vurgusu:** Düzensiz matrisin farkı yalnızca oluşturma ve iç döngünün sınırındadır. Hücreye erişim normal matristeki gibi `matrix[i][j]`'dir.

Hücrelere klavyeden değer girmemek için rastgele değerler yazıldı ve ekrana basıldı. Hoca satır sayısına 8 girdi ve her satırın farklı uzunlukta oluştuğunu gösterdi.

6.4 Serbest Bırakma

Serbest bırakma, oluşturmadan çok daha kolaydır:

```
for (i = 0; i < n; i++) {
    free(matrix[i]);
}
free(matrix);
```

Önce her `matrix[i]`, sonra `matrix` serbest bırakılır. Burada satır uzunluğunu söylemek gerekmez, çünkü `free`'ye yalnızca başlangıç adresi verilir.

6.5 Satırı Fonksiyona Göndermek

Hoca tekrar olsun diye `print_list(int *list, int size)` fonksiyonunu yazdı ve iç döngüdeki yazdırmanın yerine koydu.

Soru: Boyut olarak ne gönderilir? **Cevap:** `array[i]`, çünkü o satırın boyutu oradadır.

Soru: İşaretçi olarak ne gönderilir? **Cevap:** İlgili satırın adresi gerekir. Hoca bunu `*(*(matrix + i) + j)` gösteriminden yola çıkarak buldu. Bu ifade `i`. satırın `j`. sütunundaki değeri verir. Değerle değil satırla işimiz olduğu için `j` çıkarılır ve `*(matrix + i)` kalır. `matrix` bir adrestir, `i` eklenince yine adres olur. `*` ile o adresteki değer alınır. Bu değer de `i`. satırın başlangıç adresidir.

Aynı işi yapan daha kısa yazım `matrix[i]`'dir. Hoca bunun "sizi işaretçi yazımından kurtardığını" söyledi. İkinci parametre için de `array[i]` yerine `*(array + i)` yazılabilir:

```
print_list(matrix[i], array[i]);
print_list(*(matrix + i), *(array + i));
```

İki çağrı aynı sonucu verir.

> **Ders sonrası not:** Hoca `matrix` değişkeninin sıfırıncı satır, sıfırıncı sütunun adresini tuttuğunu söyledi. Bu dinamik matris için tam doğru değildir. `matrix` satır işaretçileri dizisinin adresini tutar, `[0][0]` hücresinin adresi ise `matrix[0]`'dadır. gcc ile denendi: `matrix` ile `matrix[0]` farklı adresler verdi.

Tam kod şöyledir. Hocanın kodunda yardımcı dizi için `free` söylenmedi, burada eklendi.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void print_list(int *list, int size) {
    int i;
    for (i = 0; i < size; i++) {
        printf("%2d ", list[i]);
    }
    printf("\n");
}

int main(void) {
    int n, i, j;
    int *array;
    int **matrix;

    printf("Satir sayisini giriniz: ");
    scanf("%d", &n);
    srand(time(NULL));

    array = (int *) malloc(n * sizeof(int));
    for (i = 0; i < n; i++) {
        array[i] = rand() % n + 1;
    }

    matrix = (int **) malloc(n * sizeof(int *));
    for (i = 0; i < n; i++) {
        matrix[i] = (int *) malloc(array[i] * sizeof(int));
    }

    for (i = 0; i < n; i++) {
        for (j = 0; j < array[i]; j++) {
```

```
            matrix[i][j] = rand() % 10;
        }
        print_list(matrix[i], array[i]);
    }

    for (i = 0; i < n; i++) {
        free(matrix[i]);
    }
    free(matrix);
    free(array);
    return 0;
}
```

Hoca kendi başınıza denemeniz için bir çalışma sorusu önerdi: Hem düzensiz hem de `realloc` ile satır boyutları değiştirilebilen iki boyutlu bir matris kodu yazın. Satır boyutları tekrar sorulsun ve bir yazdırma fonksiyonu olsun.

7. Fonksiyonlar: Sıra, Prototip ve Dizi Parametreleri

Hoca bu kısmı slaytlardan geçti, çünkü çoğu önceden görülmüştü. Programı çalıştırdığınızda C dilinde çalışan fonksiyon `main`'dir. Fonksiyonlar `main`'den önce yazılırsa prototip yazmaya gerek kalmaz. Sonra yazılacaksa prototip (function declaration) yazılır.

Dizilerin parametre olarak gönderilmesinde slaytta üç yazım vardı.

Soru: Bu üç yazımdan hangisi daha geneldir? **Cevap:** En alttaki, yani `int *a` biçimi. Hoca aralarında işlev bakımından fark olmadığını söyledi: İster dizi olduğunu belirterek yazın, ister işaretçi olarak. Fakat `int *a` daha geniş bir kavramdır: Normal bir işaretçi değişkeni de kullanabilirsiniz, illa dizi olması gerekmez.

Hoca bunu şöyle özetledi: Her dizi değişkeni aynı zamanda bir işaretçidir, ama her işaretçi aynı zamanda bir dizi değişkeni değildir.

> **Ders sonrası not:** Parametre olarak `int a[]` ile `int *a` aynıdır. gcc ile denendi: `int a[]` parametreli fonksiyona `malloc`'tan gelen işaretçi uyarısız gönderilebildi. "Her dizi bir işaretçidir" sözü ise yaklaşık bir ifadedir. Dizi adı ifadelerde ilk elemanın adresine dönüşür, ama dizinin kendisi işaretçi değildir. gcc ile denendi: `int d[5]` için `sizeof(d)` 20, `sizeof(int *)` 8 verdi.

8. Fonksiyondan Dizi Döndürme

8.1 Doğrudan Dizi Döndürülemez

Normalde bir fonksiyonun dönüş değeri dizi olamaz. Java'da ve C#'ta var, C'de yok. Aşağıdaki gibi bir yazım zaten derlenmez:

```
int[5] do_something(void);
```

Çözüm dinamik bellektir. Bir dizi döndürülecekse işaretçi döndürülür. Peki bu işaretçi nasıl olmalıdır?

8.2 Yerel Dizinin Adresini Döndürmek

Hoca şu kodu yazıp sınıfa sordu:

```
int *do_something(void) {
    int d[5];
    return d;
}
```

Kod hata vermedi, derleyici yalnızca uyarı verdi. Yazım olarak doğru, ama mantık olarak yanlıştır.

Soru: Bu dizi belleğin hangi bölgesinde oluşturulur? **Cevap:** Sınıftan hem stack hem heap cevabı geldi. Bir öğrenci, işaretçi döndürüldüğü için heap'te olması gerektiğini düşündüğünü,

yanlış ezberlediğini söyledi. Hoca bunun işaretçiyle ilgisi olmadığını belirtti: İşaretçi yalnızca dönüş değeridir. Sabit uzunluklu diziler stack'te tanımlanır. `malloc` ile oluşturulan diziler ve matrisler heap'tedir.

Bu `d` yerel bir değişkendir. Fonksiyon bitip çağrıldığı yere dönüldüğünde dizi artık yaşamaz. Eski çöp değerleri orada durabilir, ama bizim işimize yaramaz. Burada oluşturulan beş elemanlı diziye sağlıklı erişim mümkün değildir.

> **Hocanın vurgusu:** Sabit uzunluklu diziler stack'te, `malloc` ile oluşturulanlar heap'tedir. Bu ayrımın dönüş değerinin işaretçi olmasıyla ilgisi yoktur.
>
> **Ders sonrası not:** gcc ile denendi: "function returns address of local variable" uyarısı alındı ve bu derleyicide fonksiyon `NULL` döndürdü. Hocanın cümlesindeki "daima stack" yerel diziler içindir. `static int d[5]` gibi bir tanım uyarısız derlendi ve fonksiyondan sonra da değerini korudu.

8.3 Doğru Yol

```
int *do_something(void) {
    int *d = (int *) malloc(5 * sizeof(int));
    return d;
}
```

Heap'teki bu alan fonksiyon bitince de yaşar. Çağıran yer döndürülen değeri `int *p` gibi bir işaretçide saklar. Üç yazım özetle şöyledir:

- **Dönüş tipi dizi:** Derleme hatasıdır, hiç olmaz.
- **Yerel dizinin adresi:** Yazım doğru, mantık hatalıdır.
- **`malloc` ile açılıp döndürülen işaretçi:** Doğru olan budur.

8.4 İki Diziyi Birleştirme

Slayttaki örnek iki diziyi uç uca ekler. Hocanın örneğinde birinci dizi 1, 2, 3, ikinci dizi 4, 5'tir ve sonuç 1, 2, 3, 4, 5 olmalıdır. Boyutların toplamı kadar yer `malloc` ile açılır. Önce birinci dizi, ardından ikinci dizi kopyalanır ve `merge` işaretçisi döndürülür. Fonksiyon bitse bile bu alana ulaşılabilir.

```
#include <stdio.h>
#include <stdlib.h>

int *merge(int *array1, int size1, int *array2, int size2) {
    int i;
    int *merged = (int *) malloc((size1 + size2) * sizeof(int));
    if (merged == NULL) {
        return NULL;
    }
    for (i = 0; i < size1; i++) {
        merged[i] = array1[i];
    }
    for (i = 0; i < size2; i++) {
        merged[size1 + i] = array2[i];
    }
    return merged;
}

int main(void) {
    int a[] = {1, 2, 3};
    int b[] = {4, 5};
    int i;
    int *c = merge(a, 3, b, 2);

    if (c != NULL) {
        for (i = 0; i < 5; i++) {
```

```
            printf("%d ", c[i]);
        }
        printf("\n");
        free(c);
    }
    return 0;
}
```

Çıktı `1 2 3 4 5` olur (gcc ile denendi).

> **Hocanın vurgusu:** `malloc` yerine fonksiyonun içinde `int merge[size1 + size2]` yazıp aynı kodu kullanırsanız "hocam kod çalıştı" diyebilirsiniz. Çalışabilir, ama bazı zamanlarda çalışmaz, çünkü fonksiyon bitince böyle bir dizi yoktur. Böyle bir çözüm doğrudan çöptür.

8.5 İkinci Versiyon: Sonuç Dizisini Çağıran Yerde Oluşturmak

Slaytta alternatif bir yaklaşım da vardı: Fonksiyon bellek ayırmaz. Çağıran yer boş bir dizi oluşturur ve onu `array3` parametresiyle gönderir. `concatenate_array` yalnızca içini doldurur.

Soru: Bu yaklaşım gerçekten dinamik bir dizi olarak düşünülebilir mi? **Cevap:** Hoca, üç dizi de `malloc` ile oluşturulduysa kodun mantıken doğru olduğunu söyledi. Slayttaki örnekte `array1` ve `array2` sabit uzunlukluydu, `array3`'ün boyutu ise `size1 + size2`'ye göre değişiyordu. Hoca, slaytı Yunus Hoca'nın hazırladığını belirterek bu yaklaşımı doğru bulmadığını ve kabul etmeyeceklerini söyledi. Doğrusu `array3 = malloc(size3 * sizeof(int))` yazıp diziyi öyle göndermektir.

> **Ders sonrası not:** Hocanın kuralı derste geçerlidir, ama bu kodun durumu 8.4'teki hatayla aynı değildir. Burada dizi çağıran fonksiyonda yaşamaya devam eder, bu yüzden fonksiyon çalışırken geçerlidir. gcc ile `-Wall` ve AddressSanitizer açık denendi: `int c[s1 + s2]` biçimi 1, 2, 3, 4, 5 sonucunu hatasız verdi. Asıl kısıt, değişken uzunluklu dizinin eski standartta olmamasıdır: `-std=c90 -pedantic` ile "ISO C90 forbids variable length array" uyarısı alındı.

Hoca, çalışma sorusu olarak iki sıralı diziyi dinamik olarak birleştiren bir program yazmayı önerdi. Fonksiyon işaretçileri konusunun da ayrıca anlatılacağını belirtti.

9. main Fonksiyonunun Argümanları

9.1 argc ve argv

`main` de parametre alabilir. Bu parametreler, programın çalıştırılması sırasında dışarıdan verilen verileri işlemek içindir. Gelişigüzel alınamazlar. İki parametre vardır:

```
int main(int argc, char *argv[])
```

- **`argc` (argument counter):** Argüman sayısıdır.
- **`argv` (argument vector):** Argümanları tutan, karakter işaretçilerinden oluşan dizidir. Karakter dizisi string olduğuna göre bu yapı birden fazla string tutar. Asistan da önceki bir örnekte böyle bir string dizisini fonksiyona göndermişti.

  > **Hocanın vurgusu:** `main`'e verilen argümanlar otomatik olarak string türündedir. Bu dikkatten kaçmasın.

Argümanlar nereden gelir? `main` kodun içinden `main(2, ...)` diye çağrılamaz, bu `main`'in içinden `main`'i çağırmak gibi sonsuz bir şey olur. Argümanlar programı başlatırken verilir. Dev-C++'ta Execute menüsündeki Parameters kısmına yazılır. Aralarına virgül değil boşluk konur.

Hoca önce `argc`'yi, `argv[0]`'ı ve son eleman olan `argv[argc - 1]`'i yazdırdı, sonra hepsini döngüyle yazdırdı:

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    int i;
```

```
    printf("%d\n", argc);
    for (i = 0; i < argc; i++) {
        printf("%s\n", argv[i]);
    }
    return 0;
}
```

Parametre olarak `deneme abc 123` verilince `argc` 4 çıktı. Çünkü `argv[0]` her zaman programın adıdır, Dev-C++'tan çalıştırılınca tam yoluyla yazılır. Hiç argüman verilmeyince de `argc` 1 olur. Komut satırından `MainArguments deneme abc 123 deneme2` yazılırsa 4 parametre verilmiş, `argc` 5 olmuştur (gcc ile denendi).

Bu yüzden üç parametre verildiğinde üçüncüsü `argv[3]`'tür. Sıfırıncı eleman programın adına ayrılmıştır, ona dokunamayız.

9.2 Ne İşe Yarar?

Şimdiye kadar veriler hep programın içinden alındı. Oysa bir programın ürettiği veri başka bir programa verilebilir. Hoca bunu tam haberleşme olmasa da programları haberleştirmek gibi bir şey olarak anlattı. Önceden grafik arayüz olmadığı için kişi programı argümanlarla çalıştırır, çıktısını da diğer işlerinde kullanırdı.

Hoca bunu komut satırında gösterdi:

- `cd Desktop`: Bulunulan klasörü değiştirir. `Desktop` bu programa dışarıdan verilen bir parametredir.
- `dir`: Klasördeki dosyaları listeler. Linux ve Unix'te karşılığı `ls`'tir. PowerShell ikisini de destekler, `ls` yazınca arkada aynı iş çalışır.
- `dir /w`: Dosyaları kolon hâlinde gösterir.
- `dir /p`: Çok dosya varsa sayfa sayfa gösterir, devam etmek için bir tuşa basılır. Hoca bunu `C:\Windows` içinde gösterdi.

Bu komutlar parametresine göre farklı davranır. Kendi programımızda da aynısını yapabiliriz: `argv[1]`'in değerine göre bir iş, başka değerine göre başka bir iş yapılabilir. Karşılaştırma bir string üzerindeyse daha önce görülen `strcmp` kullanılır, örneğin `strcmp(argv[1], "/p") == 0`.

> **Ders sonrası not:** Hoca `cd`'yi "program" olarak anlattı. gcc ve bash ile denendi: Linux'ta `cd` kabuğun iç komutudur (`type cd`), `ls` ayrı bir programdır. Windows'ta `cd` ve `dir` de büyük olasılıkla `cmd`'nin iç komutlarıdır. Parametre verme mantığı aynıdır. Ayrıca hocanın ağızdan söylediği "`argv[1]` sıfıra eşitse" kontrolü `argv[1] == 0` diye yazılırsa sayıyı değil işaretçiyi karşılaştırır: gcc ile denendi, `0` argümanı verilince bu koşul doğru olmadı. Sayı olarak karşılaştırmak için önce `atoi` gerekir.

9.3 Kullanım Bilgisi ve Sayıya Çevirme

Hangi argümanın ne anlama geldiğini programı kullanacak kişiye bildirmeniz gerekir, yani bir kullanım kılavuzu gibi. Hocanın örneği bir sınav yeri programıdır: Program öğrenci sayısını, ders sayısını ve sınav yerini dışarıdan alır, öğrenci sayısı ile ders sayısını çarpıp toplam iş yükünü bulur.

Argüman verilmediyse (`argc == 1`) şu bilgi yazılır: Program yanlış başlatıldı, en az 3 argüman girilmeli. Argüman 1 öğrenci sayısı, argüman 2 ders sayısı, argüman 3 sınav yeridir.

Hoca önce çarpmayı doğrudan yazdı:

```
yuk = argv[1] * argv[2];
```

Derleyici "invalid operands" hatası verdi, çünkü iki `char *` çarpılamaz. Argümanlar sayı olarak kullanılacaksa çevrilmelidir. `atoi` tam sayıya, `atof` `double`'a çevirir. Bu tür dönüşüm fonksiyonları çok fazla değildir, üç dört tane kadardır.

> **Ders sonrası not:** Hoca "artı için yapsan hiç sıkıntı yok" dedi, ardından iki stringin toplandığı dillerden söz etti. C'de `argv[1] + argv[2]` de derlenmez. gcc ile denendi: "invalid operands to binary +" hatası alındı. Ayrıca hoca `atoi`'yi "argument to integer" diye açıkladı; adındaki `a` büyük olasılıkla ASCII'den gelir.

Düzeltilmiş kod şöyledir:

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int yuk;
    if (argc < 4) {
        printf("Program yanlis baslatildi.\n");
        printf("En az 3 arguman girilmelidir.\n");
        printf("Argumanlarin anlamlari:\n");
        printf("Arguman 1: ogrenci sayisi\n");
        printf("Arguman 2: ders sayisi\n");
        printf("Arguman 3: sinav yeri\n");
    } else {
        yuk = atoi(argv[1]) * atoi(argv[2]);
        printf("Sinav yeri: %s\n", argv[3]);
        printf("Toplam yuk: %d\n", yuk);
    }
    return 0;
}
```

> **Ders sonrası not:** Derste yalnızca `argc == 1` kontrol edildi. Program `argv[3]`'e eriştiği için en az 3 argüman gerekir. Yukarıdaki kodda bu yüzden `argc < 4` yazıldı. Aksi hâlde 1-2 argümanla çalıştırılınca var olmayan bir argümana erişilirdi.

9.4 Çalışma Zamanı Hatası

Soru: Bu çarpmada program çalışırken hata meydana gelebilir mi? **Cevap:** Bir öğrenci, sayıya dönüştürülemeyen bir şey yazılırsa olur dedi. Hoca bunu denedi. Sonuçlar şunlardır (`atoi` değerleri gcc ile, çarpımlar python ile doğrulandı):

1. argüman	2. argüman	3. argüman	`atoi(argv[1])`	Toplam yük
`123`	`5`	`D114`	123	$123 \times 5 = 615$
`5B`	`5`	`D114`	5	$5 \times 5 = 25$
`B5`	`5`	`D114`	0	0
`abcd`	`5`	`D114`	0	0

Hoca, `B5` ve `abcd` için `atoi`'nin başarısız olup 0 döndürdüğünü, çarpımın da bu yüzden 0 çıktığını gösterdi. Bu fonksiyonların kullanımı basittir, ama kullanacak kişinin kurallara uyması gerekir. Hoca aynı denemeyi `cmd` üzerinden de yaptı.

> **Ek bilgi (derste anlatılmadı):** `atoi`, gerçek 0 değeri ile başarısız dönüşümü ayırt edemez. `strtol` çevrilemeyen kısmın adresini de verdiği için bu ayrım yapılabilir. gcc ile denendi: `strtol("5B", &son, 10)` 5 ve kalan `B`, `strtol("B5", &son, 10)` 0 ve kalan `B5` verdi.

9.5 Komut Satırı Seçenekleri

Hoca son olarak, komut satırı argümanlarını daha uygun kullanmayı sağlayan fonksiyonlardan kısaca söz etti (`getopt` ve benzeri). Bunlara `argc`, `argv` ve bir seçenek metni verilir. Bu metin hangi seçeneklerin kabul edileceğini söyler, örneğin A, B, C ve D, ve bir seçenek gerekli (mandatory) yapılabilir. Hocaya göre bunlar arka planda bizim `if`'lerle yaptığımız kontrolleri yapar, her yerde desteklenmeyebilir ve büyük olasılıkla Linux üzerinde çalışır. Örneği bulursa paylaşacağını söyledi.

> **Ek bilgi (derste anlatılmadı):** `getopt`'ta seçenek metnindeki iki nokta, seçeneğin zorunlu olduğunu değil değer istediğini belirtir. gcc ile Linux'ta denendi: metin `"abcd:"` iken `-d x` değeri okudu, değersiz `-d` hata verdi, ama `-d` hiç yazılmadığında hata alınmadı.

Dersin Toparlaması

Uygulamada dinamik matris, kapasiteyi iki katına çıkararak büyüyen dizi, dinamik string dizisi ve iç içe döngüyle eşleştirme görüldü. Üç sorunun ortak yapısı aynıydı: Belleği ayır, doldur, yazdır, ayırma sırasının tersiyle serbest bırak.

Ek derste hoca bu araçların doğru kullanımına odaklandı. `realloc` sonucu önce geçici işaretçide kontrol edilmeli, düzensiz matriste iç döngünün sınırı o satırın kendi boyutu olmalı, fonksiyondan yerel dizinin adresi değil `malloc` ile ayrılmış alanın adresi döndürülmeli, `main` argümanları string olduğu için `atoi` ve `atof` ile çevrilmelidir. Hoca çarşamba günü `struct`'larla devam edileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.